CE49X
Introduction to Data Science for Civil Engineering

Final Project

Conflict Situation Monitoring for Maritime Shipping:
Correlating Satellite Thermal Anomalies with War-Related Events

Boğaziçi University

Spring 2026

Dr. Eyuphan Koc

April 17, 2026

Contents

1 Project Overview **3**
1.1 Scenario . . . 3
1.2 Core Question . . . 3
1.3 Project Approach . . . 3
1.4 Learning Objectives . . . 3

2 Team Formation **4**

3 Background: NASA FIRMS **4**
3.1 Accessing the Data . . . 4
3.2 FIRMS Data Fields . . . 4

4 Task 1: Data Collection & Assembly **4**
4.1 Objective . . . 4
4.2 NASA FIRMS Thermal Data . . . 5
4.3 War & Conflict News Collection . . . 5
4.4 Deliverables for Task 1 . . . 6

5 Task 2: Spatial & Temporal Analysis **6**
5.1 Objective . . . 6
5.2 Thermal Event Clustering . . . 6
5.3 Temporal Patterns . . . 7
5.4 Spatial Analysis . . . 7
5.5 Deliverables for Task 2 . . . 7

6 Task 3: Thermal–News Correlation & Classification **7**
6.1 Objective . . . 7
6.2 Thermal–News Matching . . . 8
6.3 Regional News Reporting Comparison . . . 8
6.4 Predicting Conflict Association (ML Classification) . . . 8
6.5 Deliverables for Task 3 . . . 9

7 Task 4: Dashboard, Insights & Reflection **9**
7.1 Objective . . . 9
7.2 Multi-Panel Summary Dashboard . . . 9
7.3 Written Discussion & Industry Implications . . . 10
7.4 Deliverables for Task 4 . . . 10

8 Technical Requirements **10**
8.1 Programming Languages and Tools . . . 10

9 Database Storage with PostgreSQL & Docker **11**
9.1 Setting Up PostgreSQL with Docker . . . 11
9.2 Connecting from Python . . . 11
9.3 Required Tables . . . 12
9.4 Verification . . . 12

10 Final Deliverables **12**
10.1 Code Repository 12
10.2 Project Notebook 12
10.3 Dashboard Figure 12
10.4 Final Presentation 13

11 Grading Summary **13**

1 Project Overview

1.1 Scenario

You are the data analytics team at a **maritime shipping company**. Your company operates cargo vessels across global trade routes, and your operating costs are heavily tied to **oil and gas prices**. When armed conflicts erupt—especially in energy-producing or strategically important regions—fuel prices spike, shipping routes get disrupted, and insurance premiums surge.

Your management wants an **early situation monitoring system**: a data pipeline that detects thermal anomalies from satellite imagery in conflict-prone regions around the world and cross-references them with war and conflict news to assess whether these hotspots are associated with armed conflict activity.

1.2 Core Question

> *Can satellite-detected thermal anomalies, when correlated with conflict-related news, serve as indicators of armed conflict activity in regions that matter to global shipping and energy markets?*

1.3 Project Approach

This project follows a situation monitoring pipeline:

1. **Data Collection:** Acquiring worldwide satellite thermal data from NASA FIRMS and collecting war and conflict news from web sources.
2. **Spatial-Temporal Analysis:** Clustering thermal detections into discrete hotspot events, analyzing their geographic and temporal patterns across conflict-prone regions.
3. **Correlation & Classification:** Matching thermal events with conflict news, analyzing whether conflict zones exhibit distinct thermal signatures, and building ML models to classify thermal anomalies.

1.4 Learning Objectives

By completing this project, students will:

- Independently research and identify relevant data sources for a real-world monitoring problem.
- Collect and integrate heterogeneous data—structured satellite data (API) and unstructured text data (web scraping).
- Apply geospatial analysis at a global scale: spatial clustering, cross-region comparison, and geographic visualization.
- Perform statistical hypothesis testing to compare groups and interpret significance.
- Build and evaluate machine learning classifiers on a novel prediction problem.
- Create publication-quality multi-panel visualizations for a decision-making audience.
- Think critically about data quality, correlation vs. causation, and analysis limitations.
- Connect data science methods to real-world industry applications in logistics, risk assessment, and energy markets.

2 Team Formation

- **Group Size:** 1–2 students per group
- **Collaboration:** All team members must contribute equally to the project
- **Deliverables:** Each group submits one final notebook and code repository

3 Background: NASA FIRMS

NASA's **Fire Information for Resource Management System (FIRMS)** provides near real-time active fire and thermal anomaly data derived from satellite observations. The system uses thermal anomaly detection algorithms applied to data from two instrument families:

- **MODIS** (Moderate Resolution Imaging Spectroradiometer) — aboard the Terra and Aqua satellites, providing data at $\sim$1 km resolution since 2000.
- **VIIRS** (Visible Infrared Imaging Radiometer Suite) — aboard the Suomi NPP and NOAA-20 satellites, providing data at $\sim$375 m resolution since 2012.

Each detection record includes geographic coordinates, acquisition date and time, confidence level, brightness temperature, and **Fire Radiative Power (FRP)**—a measure of the thermal energy output in megawatts. Importantly, FIRMS detects *any* thermal anomaly, not just wildfires—this includes industrial fires, oil facility burns, explosions, and other high-temperature events.

3.1 Accessing the Data

1. Register for a free MAP_KEY at: `https://firms.modaps.eosdis.nasa.gov/api/area/`
2. Use the **area endpoint** (bounding box) or the **country endpoint** (ISO3 country codes) to download data.
3. The API returns up to **5 days per request**. For historical data, append a date parameter (`YYYY-MM-DD`). To collect longer periods, loop through 5-day chunks.
4. Use `_SP` (Standard Processing) sources for historical data and `_NRT` for near real-time.

Full API documentation: `https://firms.modaps.eosdis.nasa.gov/api/area/`

3.2 FIRMS Data Fields

4 Task 1: Data Collection & Assembly

Points: 30/100

4.1 Objective

Build a multi-source dataset for conflict situation monitoring: satellite thermal anomaly data from NASA FIRMS and war/conflict-related news articles.

Field	Description	Example
`latitude`	Latitude of detection pixel center	33.312
`longitude`	Longitude of detection pixel center	44.361
`brightness`	Brightness temperature (Kelvin)	328.5
`acq_date`	Acquisition date	2024-08-15
`acq_time`	Acquisition time (HHMM UTC)	1342
`satellite`	Satellite name	Suomi NPP
`confidence`	Detection confidence	nominal / high
`frp`	Fire Radiative Power (MW)	12.4
`daynight`	Day or night observation	D / N

Table 1: Key fields in FIRMS active fire / thermal anomaly data.

4.2 NASA FIRMS Thermal Data

- Register for a free FIRMS MAP_KEY.
- Select your regions of interest that are relevant to conflict monitoring and global shipping. Justify your selection.
- Collect VIIRS and/or MODIS data for your selected regions covering a continuous period of **at least 6 months**.
- Minimum **5,000 thermal detection points** across all regions combined.
- Load into a Pandas DataFrame. Show `df.head()`, `df.describe()`, and check for missing values.
- Clean the data:
  - Parse `acq_date` to a proper datetime type.
  - Filter out low-confidence detections (document the threshold you choose and justify it).
  - Add a column identifying which region each detection belongs to.
  - Handle any missing or anomalous values.
- Document: API endpoint(s) used, bounding boxes or country codes, date range, satellite instrument(s), and total records before and after cleaning.

4.3 War & Conflict News Collection

- Collect at least **1,000 war/conflict-related news articles** covering your selected regions during the same time period as your FIRMS data.
- Use **at least 2 different sources**. Some options:
  - **News APIs:** NewsAPI, GNews, MediaStack, TheNewsAPI
  - **Aggregators:** Google News (RSS feeds or scraping), Bing News
  - **International Outlets:** BBC, Reuters, Al Jazeera, Associated Press, CNN, The Guardian
  - **Conflict-Focused Sources:** The War Zone, Defense News, Liveuamap, Conflict News
  - **Wire Services:** AFP, DPA, Anadolu Agency

- Search queries should include conflict-related keywords such as: "war", "conflict", "military", "bombing", "airstrike", "shelling", "missile", "attack", "troops", "armed", "explosion", "combat".
- For each article, extract:
  - Title
  - Publication date
  - Source / Publisher
  - URL
  - Location mention(s) (country, city, or region name from the headline or snippet)
- Store in a structured Pandas DataFrame with consistent column names.

4.4 Deliverables for Task 1

- Data collection scripts (API calls, scraping code).
- Clean DataFrames loaded and explored (`head()`, `describe()`, `info()`).
- Markdown cells documenting each source: URL, access date, method, and record count.
- A written justification of your region selection (why these regions matter for shipping and energy).

5 Task 2: Spatial & Temporal Analysis

Points: 25/100

5.1 Objective

Transform raw satellite detection points into meaningful **thermal events**, then analyze their geographic distribution, temporal patterns, and regional characteristics.

5.2 Thermal Event Clustering

Raw FIRMS data contains individual satellite observation pixels. Multiple detections that are close in space and time typically represent a single thermal event. Your task is to cluster these into discrete **thermal events**.

- **Grouping approach:** Cluster detections that fall within a spatial radius (suggested: 5–10 km) and temporal window (suggested: 1–3 days) into one thermal event.
  - *Simple approach:* Group by region/country and date, then merge overlapping date windows.
  - *Advanced approach:* Use DBSCAN from scikit-learn with a Haversine distance metric on (lat, lon, time) features.
- For each thermal event, compute:
  - Centroid coordinates (mean latitude, mean longitude)
  - Start date and end date
  - Duration (number of days active)

  - Total FRP (sum of all detections)
  - Maximum brightness temperature
  - Number of satellite detection points
  - Region label

- Report the total number of discrete thermal events identified per region and describe your clustering methodology.

5.3 Temporal Patterns

- Create a **monthly time series** of thermal event frequency, **broken down by region**. Are there temporal spikes that could correspond to conflict escalation?
- Analyze **day vs. night** detection patterns using the `daynight` column. Do conflict-prone regions show different day/night ratios than others?
- Compare thermal intensity (mean FRP) across regions and over time.
- Produce at least **2 temporal visualizations** (e.g., monthly event count by region, FRP trend over time).

5.4 Spatial Analysis

- Create a **world map** (or multi-region map) showing all thermal events:
  - Marker size proportional to total FRP.
  - Color mapped to region or duration.
  - Use a proper sequential colormap (**not** rainbow or jet).
- Identify and label the **top hotspot clusters** within each region by number of events or total FRP.
- Produce at least **2 spatial visualizations**.

5.5 Deliverables for Task 2

- Thermal event clustering code with documented methodology and parameter choices.
- Summary DataFrame of thermal events with all computed features.
- At least 4 publication-quality visualizations (2 temporal, 2 spatial) with axis labels, titles, legends, and colorbars where applicable.

6 Task 3: Thermal–News Correlation & Classification

Points: 30/100

6.1 Objective

Match thermal events with conflict news, analyze whether conflict-active regions show distinct thermal signatures, and build machine learning models to classify thermal anomalies.

6.2 Thermal–News Matching

Develop a strategy to link conflict news articles to thermal events:

- **Temporal matching:** The article was published within a reasonable window of the thermal event (e.g., 0–7 days).
- **Spatial/location matching:** Location mentions in the article (country, city, region) correspond to the thermal event's geographic area.
- **Keyword relevance:** The article contains conflict-related keywords confirming it is about armed conflict activity.
- For each thermal event, determine:
  - **Conflict-associated:** At least one matching conflict news article found.
  - **No conflict association:** No matching articles (likely a natural fire, industrial activity, or agricultural burn).
- Report the **conflict-association rate** per region: what fraction of thermal events in each region have matching conflict news?

6.3 Regional News Reporting Comparison

Different conflict zones receive very different levels of media attention. Analyze how news reporting varies across your selected regions:

- For each region, compute: total number of articles, number of unique sources covering it, average articles per thermal event, and the time difference between satellite detection and the earliest news article.
- Compare regions: which conflict zones are heavily covered and which are underreported?
- Perform a **statistical hypothesis test** comparing a metric of your choice (e.g., FRP, reporting delay, article count) between regions or between conflict-associated and non-conflict thermal events. State hypotheses, report the test statistic and p-value, and interpret the result.
- Create at least **2 visualizations** comparing coverage across regions (e.g., stacked bar chart of article count by region and source, heatmap of keyword frequency by region).
- Discuss: which regions are "blind spots" where satellite data is especially valuable because news coverage is sparse?

6.4 Predicting Conflict Association (ML Classification)

Build a classifier to predict whether a thermal event is conflict-associated:

- **Features:** Total FRP, duration (days), max brightness, number of detection points, day/night ratio, latitude, longitude, region, month or season.
- **Target:** Conflict-associated (1) vs. Non-conflict (0).
- **Pipeline:**
  - Train/test split (80/20, stratified by target).
  - Feature scaling with `StandardScaler` (fit on training set only).

  - Train at least **2 classifiers** from: Logistic Regression, Decision Tree, Naive Bayes, SVM.

- **Evaluation:**
  - Report accuracy, precision, recall, and F1-score for each model.
  - Display the confusion matrix as a heatmap.
  - Discuss: in the context of a shipping company's risk monitoring, is it worse to miss a real conflict (false negative) or to raise a false alarm (false positive)? How does this affect your choice of evaluation metric?

- Identify the **most predictive features** (e.g., via logistic regression coefficients or decision tree feature importances). What does this reveal about the characteristics of conflict-related thermal anomalies?

6.5 Deliverables for Task 3

- Matching algorithm code with clearly documented logic and parameter choices.
- Conflict-association rates per region.
- Regional news comparison with hypothesis test, coverage statistics, and blind spot discussion.
- ML classification pipeline with evaluation metrics and confusion matrices.
- At least 5 visualizations across all sub-tasks.

7 Task 4: Dashboard, Insights & Reflection

Points: 15/100

7.1 Objective

Synthesize all findings into a multi-panel dashboard and a written discussion connecting the results to real-world decision-making in maritime shipping and energy risk management.

7.2 Multi-Panel Summary Dashboard

Create a single multi-panel figure using `matplotlib.gridspec.GridSpec`:

- At least **4 panels** with at least **3 different chart types**.
- Must include:
  1. A world/multi-region map showing thermal events color-coded by conflict association.
  2. A conflict-association rate comparison across regions.
  3. A temporal trend or reporting pattern visualization.
  4. One panel of your choice highlighting a key finding.
- At least one panel should span multiple grid cells.
- Include an overall `fig.suptitle()` that communicates the main finding as a headline.
- Save the figure as `dashboard.png` at 300 DPI using `plt.savefig()`.

7.3 Written Discussion & Industry Implications

Answer all of the following in separate markdown cells (minimum 1 paragraph each):

1. **Key Findings:** What did you discover about the relationship between satellite thermal anomalies and conflict activity? Which regions showed the strongest signal? What surprised you?
2. **Shipping & Energy Implications:** Based on your analysis, which regions pose the highest risk for shipping route disruptions and energy price volatility? If you were presenting to your company's management, what actionable recommendations would you make for route planning and risk hedging?
3. **Limitations & Future Work:** What are the main limitations of your analysis? Consider: matching accuracy, the inability to distinguish conflict fires from natural fires purely by satellite data, temporal coverage, news bias toward certain regions, and confounding factors. What additional data or methods would improve the system?
4. **Methodology Reflection:** Which part of the data science pipeline was most challenging? What would you do differently if you started over?

7.4 Deliverables for Task 4

- Multi-panel dashboard figure saved as `dashboard.png`.
- Written discussion (4 sections, minimum 1 paragraph each).

8 Technical Requirements

8.1 Programming Languages and Tools

- **Primary Language:** Python 3.8+
- **Required Libraries:**
  - Data Collection: `requests`, `BeautifulSoup` (`bs4`), optionally `selenium`.
  - Data Manipulation: `pandas`, `numpy`.
  - Visualization: `matplotlib`, `seaborn`.
  - Statistics: `scipy.stats`.
  - Machine Learning: `scikit-learn` (train_test_split, StandardScaler, LogisticRegression, DecisionTreeClassifier, GaussianNB, SVC, confusion_matrix, classification_report, cross_val_score).
  - Database: `sqlalchemy`, `psycopg2-binary`.
- **Optional Libraries (not required):**
  - `folium` — interactive map visualizations.
  - `geopy` — geocoding location names to coordinates.
  - `geopandas` — geographic data with shapefiles.
- **Environment:** Jupyter Notebooks are required for this project.

9 Database Storage with PostgreSQL & Docker

All collected and processed data must be stored in a **PostgreSQL** database running inside a **Docker** container. This mirrors real-world data engineering workflows where data pipelines persist results to a database rather than flat files.

9.1 Setting Up PostgreSQL with Docker

1. **Install Docker Desktop** from `https://www.docker.com/products/docker-desktop/` (available for macOS, Windows, and Linux). After installation, verify it is running:

```
docker --version
```

2. **Pull and run the PostgreSQL image.** Open a terminal and run:

```
docker run --name ce49x-postgres \
  -e POSTGRES_USER=ce49x \
  -e POSTGRES_HOST_AUTH_METHOD=trust \
  -e POSTGRES_DB=conflict_monitoring \
  -p 5432:5432 \
  -d postgres:16
```

   This creates a container named `ce49x-postgres` with:

   - Database name: `conflict_monitoring`
   - Username: `ce49x`
   - No password required (local development only)
   - Accessible on `localhost:5432`

3. **Verify the container is running:**

```
docker ps
```

4. **To stop and restart** the container later:

```
docker stop ce49x-postgres
docker start ce49x-postgres
```

9.2 Connecting from Python

Use `sqlalchemy` to connect to the database and write DataFrames directly:

```python
from sqlalchemy import create_engine
import pandas as pd

# Create connection engine
engine = create_engine(
    "postgresql://ce49x@localhost:5432/conflict_monitoring"
)

# Write a DataFrame to a table
df_firms.to_sql("firms_detections", engine, if_exists="replace", index=False)
df_news.to_sql("news_articles", engine, if_exists="replace", index=False)

# Read back from the database
df = pd.read_sql("SELECT * FROM firms_detections LIMIT 10", engine)
print(df)
```

Table Name	Contents
`firms_detections`	Raw FIRMS thermal detection records (cleaned)
`news_articles`	Collected conflict news articles with metadata
`thermal_events`	Clustered thermal events with computed features
`event_matches`	Thermal event–news article matching results

Table 2: Required database tables.

9.3 Required Tables

You must create at least the following tables in your database:
As you progress through the tasks, insert your processed DataFrames into the corresponding tables using `to_sql()`. When performing analysis in later tasks, **read the data back from the database** using `pd.read_sql()` rather than from local variables or CSV files. This demonstrates the full pipeline: collect → store → query → analyze.

9.4 Verification

You can verify your tables directly from the terminal:

```
docker exec -it ce49x-postgres psql -U ce49x -d conflict_monitoring -c "\dt"
```

This lists all tables in the database. You can also run SQL queries:

```
docker exec -it ce49x-postgres psql -U ce49x -d conflict_monitoring \
  -c "SELECT COUNT(*) FROM firms_detections;"
```

10 Final Deliverables

10.1 Code Repository

- Well-organized GitHub repository (pushed to your course fork).
- `requirements.txt` or `environment.yml` file.
- `README.md` explaining project structure, how to set up the database with Docker, and how to reproduce the analysis.
- The `docker run` command (or a `docker-compose.yml` file) needed to recreate the database.

10.2 Project Notebook

- A single Jupyter Notebook containing all code, analysis, and written discussion.
- The notebook must **run top-to-bottom without errors** (Kernel → Restart & Run All).
- All visualizations must be publication-quality: axis labels with units, descriptive titles, legends, and appropriate colormaps.
- All data sources must be cited with URLs and access dates.

10.3 Dashboard Figure

- `dashboard.png` saved at 300 DPI alongside the notebook.

10.4 Final Presentation

Length: 10–15 minutes

- 5–7 minutes: main findings and methodology
- 3–5 minutes: Q&A
- 1–2 minutes: conclusions and future outlook

11 Grading Summary

Task	Description	Points
Task 1	Data Collection & Assembly	30
	NASA FIRMS Thermal Data	15
	War & Conflict News Collection	15
Task 2	Spatial & Temporal Analysis	25
	Thermal Event Clustering	10
	Temporal Patterns	7
	Spatial Analysis	8
Task 3	Thermal–News Correlation & Classification	30
	Matching Algorithm	10
	Regional News Reporting Comparison	15
	ML Classification	5
Task 4	Dashboard, Insights & Reflection	15
	Multi-Panel Dashboard	8
	Written Discussion	7
Total		**100**

Table 3: Point allocation by task and sub-task.